

Chapter 5

Analysis of imputed data

You must use a computer to do data science; you cannot do it in your head, or with pencil and paper.

Hadley Wickham

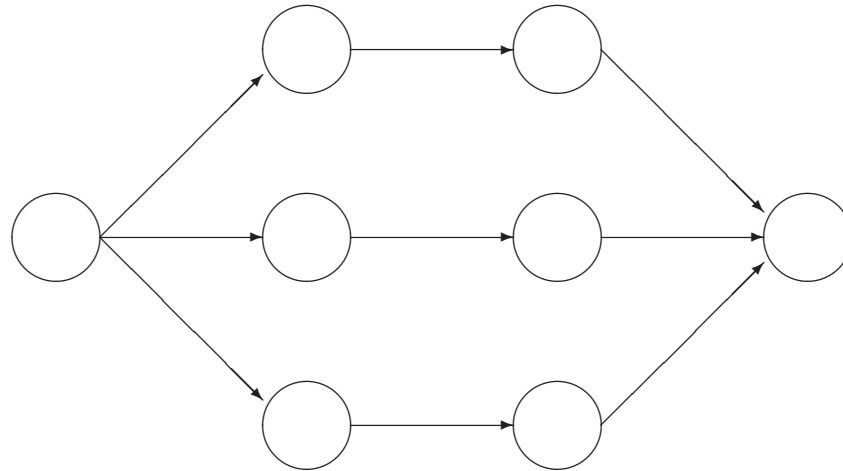
Creating plausible imputations is the most challenging activity in multiple imputation. Once we have the multiply imputed data, we can estimate the parameters of scientific interest from each of the m imputed datasets, but now without the need to deal with the missing data, as all data are now complete. These repeated analyses produce m results.

The m results will feed into step 3 (pooling the results). The pooling step to derive the final statistical inferences is relatively straightforward, but its application in practice is not entirely free of problems. First of all, the complete-data analyses are nontrivial. Historically, the imputation literature (including the first edition of this book) has concentrated on step 1 (creating the imputations) and on step 3 (pooling the results), and has worked from the notion that step 2 (estimating the parameters) is well-specified and easy to execute once the data are complete. In practice step 2 can be quite involved. The step often includes model searching, optimization, validation, prediction, assessment of the quality of model fit, in fact, step 2 may embrace almost any aspect of machine learning and data science. All the analyses need to be repeated for each of the m datasets, which may put a considerable burden on the data analyst.

Fortunately, thanks to tremendous advances in recent computational technology, the use of modern data science techniques in step 2 is now becoming feasible. This chapter focuses on step 2. The next chapter addresses issues related to step 3.

5.1 Workflow

Figure 5.1 outlines the three main steps in any multiple imputation analysis. In step 1, we create several m complete versions of the data by replacing the missing values by plausible data values. The task of step 2 is to estimate the parameters of scientific or commercial interest from each imputed



Incomplete data Imputed data Analysis results Pooled results

Figure 5.1: Scheme of main steps in multiple imputation.

Table 5.1: Overview of the classes in the `mice` package.

Class	Name	Produced by	Description
<code>mids</code>	<code>imp</code>	<code>mice()</code>	multiply imputed dataset
<code>mild</code>	<code>idl</code>	<code>complete()</code>	multiply imputed list of data
<code>mira</code>	<code>fit</code>	<code>with()</code>	Multiple imputation repeated analyses
<code>mipo</code>	<code>est</code>	<code>pool()</code>	Multiple imputation pooled results

dataset. Step 3 involves pooling the m parameter estimates into one estimate, and obtaining an estimate of its variance. The results allow us to arrive at valid decisions from the data, accounting for the missing data and having the correct type I error rate.

5.1.1 Recommended workflows

There is more than one way to divide the work that implements the steps of Figure 5.1. The classic workflow in `mice` runs functions `mice()`, `with()` and `pool()` in succession, each time saving the intermediate result:

```
# mids workflow using saved objects
library(mice)
imp <- mice(nhanes, seed = 123, print = FALSE)
fit <- with(imp, lm(chl ~ age + bmi + hyp))
est1 <- pool(fit)
```

The objects `imp`, `fit` and `est` have classes `mids`, `mira` and `mipo`, respectively. See [Table 5.1](#) for an overview. The classic workflow works because `mice` contains a `with()` function that understands how to deal with a `mids`-object. The classic `mids` workflow has been widely adopted, but there are more possibilities.

The `magrittr` package introduced the pipe operator to R. This operator removes the need to save and reread objects, resulting in more compact and better readable code:

```
# mids workflow using pipes
library(magrittr)
est2 <- nhanes %>%
  mice(seed = 123, print = FALSE) %>%
  with(lm(chl ~ age + bmi + hyp)) %>%
  pool()
```

The `with()` function handles two tasks: to fill in the missing data and to analyze the data. Splitting these over two separate functions provided the user easier access to the imputed data, and hence is more flexible. The following code uses the `complete()` function to save the imputed data as a list of dataset (i.e., as an object with class `mild`), and then executes the analysis on each dataset by the `lapply()` function.

```
# mild workflow using base::lapply
est3 <- nhanes %>%
  mice(seed = 123, print = FALSE) %>%
  mice::complete("all") %>%
  lapply(lm, formula = chl ~ age + bmi + hyp) %>%
  pool()
```

If desired, we may extend the `mild` workflow by recycling through multiple arguments by means of the `Map` function.

```
# mild workflow using pipes and base::Map
est4 <- nhanes %>%
  mice(seed = 123, print = FALSE) %>%
  mice::complete("all") %>%
  Map(f = lm, MoreArgs = list(f = chl ~ age + bmi + hyp)) %>%
  pool()
```

RStudio has been highly successful with the introduction of the free and open `tidyverse` ecosystem for data acquisition, organization, analysis, visualization and reproducible research. The book by Wickham and Golemund (2017) provides an excellent introduction to data science using `tidyverse`. The `mild` workflow can be written in `tidyverse` as

```
# mild workflow using purrr::map
library(purrr)
est5 <- nhanes %>%
  mice(seed = 123, print = FALSE) %>%
  mice::complete("all") %>%
  map(lm, formula = chl ~ age + bmi + hyp) %>%
  pool()
```

Manipulating the imputed data is easy if we store the imputed data in long format.

```
# long workflow using base::by
est6 <- nhanes %>%
  mice(seed = 123, print = FALSE) %>%
  mice::complete("long") %>%
  by(as.factor($.imp), lm, formula = chl ~ age + bmi + hyp) %>%
  pool()
```

The long format can be processed by the `dplyr::do()` function into a list-column and pooled, as follows:

```
# long workflow using a dplyr list-column
library(dplyr)
est7 <- nhanes %>%
  mice(seed = 123, print = FALSE) %>%
  mice::complete("long") %>%
  group_by(.imp) %>%
  do(model = lm(formula = chl ~ age + bmi + hyp, data = .)) %>%
  as.list() %>%
  .[[1]] %>%
  pool()
```

These workflows yield identical estimates, but allow for different extensions.

5.1.2 Not recommended workflow: Averaging the data

Researchers are often tempted to average the multiply imputed data, and analyze the averaged data as if it were complete. This method yields incorrect standard errors, confidence intervals and p -values, and thus should not be used if any form of statistical testing or uncertainty analysis is to be done on the imputed data. The reason is that the procedure ignores the between-imputation variability, and hence shares all the drawbacks of single imputation. See [Section 1.3](#).

Averaging the data and analyzing the aggregate is easy to do with `dplyr`:

```
# incorrect workflow: averaging data, no pooling
ave <- nhanes %>%
  mice(seed = 123, print = FALSE) %>%
  mice::complete("long") %>%
  group_by(.id) %>%
  summarise_all(.funs = mean) %>%
  select(-.id, -.imp)
est8 <- lm(formula = chl ~ age + bmi + hyp, data = ave)
```

This workflow is faster and easier than the methods in [Section 5.1.1](#), since there is no need to replicate the analyses m times. In the words of Dempster and Rubin (1983), this workflow is

... seductive because it can lull the user into the pleasurable state of believing that the data are complete after all.

The ensuing statistical analysis does not know which data are observed and which are missing, and treats all data values as real, which will underestimate the uncertainty of the parameters. The reported standard errors and p -values after data-averaging are generally too low. The correlations between the variables of the averaged data will be too high. For example, the correlation matrix is the average data

```
cor(ave)
```

	age	bmi	hyp	chl
age	1.000	-0.4079	0.5262	0.478
bmi	-0.408	1.0000	0.0308	0.313
hyp	0.526	0.0308	1.0000	0.381
chl	0.478	0.3127	0.3812	1.000

are more extreme than the average of the m correlation matrices¹

```
cor <- nhanes %>%
  mice(seed = 123, print = FALSE) %>%
  mice::complete("all") %>%
  lapply(cor)
Reduce("+", cor) / length(cor)
```

	age	bmi	hyp	chl
age	1.000	-0.3676	0.4741	0.442
bmi	-0.368	1.0000	0.0377	0.278
hyp	0.474	0.0377	1.0000	0.299
chl	0.442	0.2782	0.2985	1.000

¹Section 5.2.2 describes a better way to combine correlations.

which is an example of ecological fallacy. As researchers tend to like low p -values and high correlations, there is a cynical reward for the analysis of the average data. However, analysis of the average data cannot give a fair representation of the uncertainties associated with the underlying data, and hence is not recommended.

5.1.3 Not recommended workflow: Stack imputed data

A variation on this theme is to stack the imputed data, thus creating $m \times n$ complete records. Each record is weighted by a factor $1/m$, so that the total sample size is equal to n . The statistical analysis amounts to performing a weighted linear regression. If the scientific interest is restricted to point estimates and if the complete-data model is linear, this analysis of the stacked imputed data will yield unbiased estimates. Be aware that routine methods for calculating test statistics, confidence intervals or p -values will provide invalid answers if applied to the stacked imputed data.

Creating and analyzing a stacked imputed dataset is easy:

```
est9 <- nhanes2 %>%
  mice(seed = 123, print = FALSE) %>%
  mice::complete("long") %>%
  lm(formula = chl ~ age + bmi + hyp)
```

While the estimated regression coefficients are unbiased, we cannot trust the standard errors, t -values and so on. An advantage of stacking over averaging is that it is easier to analyze categorical data. Although stacking can be useful in specific contexts, like variable selection, in general it is not recommended.

5.1.4 Repeated analyses

The appropriate way to analyze multiply imputed data is to fit the complete-data model on each imputed dataset separately. In `mice` we can use the `with()` command for this purpose. This function takes two main arguments. The first argument of the call is a `mids` object produced by `mice()`. The second argument is an expression that is to be applied to each completed dataset. The `with()` function implements the following loop ($\ell = 1, \dots, m$):

1. it creates the ℓ^{th} imputed dataset
2. it runs the expression on the imputed dataset
3. it stores the result in the list `fit$analyses`

For example, we fit a regression model to each dataset and print out the estimates from the first and second completed datasets by

```
fit <- with(imp, lm(chl~bmi+age))
coef(fit$analyses[[1]])
```

(Intercept)	bmi	age
33.55	4.08	27.37

```
coef(fit$analyses[[2]])
```

(Intercept)	bmi	age
-51.47	6.77	35.44

Note that the estimates differ from each other because of the uncertainty created by the missing data. Applying the standard pooling rules is done by

```
est <- pool(fit)
summary(est)
```

	estimate	std.error	statistic	df	p.value
(Intercept)	6.26	64.29	0.0974	9.62	0.92394
bmi	4.99	2.10	2.3762	9.24	0.03370
age	29.17	9.15	3.1890	12.89	0.00718

which shows the correct estimates after multiple imputation.

Any R expression produced by `expression()` can be evaluated on the multiply imputed data. For example, suppose we want to calculate the difference in frequencies between categories 1 and 2 of `hyp`. This is conveniently done by the following statements:

```
expr <- expression(freq <- table(hyp), freq[1] - freq[2])
fit <- with(imp, eval(expr))
unlist(fit$analyses)
```

1	1	1	1	1
9	9	11	15	15

All the major software packages nowadays have ways to execute the m repeated analyses to the imputed data.

5.2 Parameter pooling

5.2.1 Scalar inference of normal quantities

Section 2.4 describes Rubin's rules for pooling the results from the m complete-data analyses. These rules are based on the assumption that the

Table 5.2: Suggested transformations toward normality for various types of statistics. The transformed quantities can be pooled by Rubin's rules.

Statistic	Transformation	Source
Correlation	Fisher z	Schafer (1997)
Odds ratio	Logarithm	Agresti (1990)
Relative risk	Logarithm	Agresti (1990)
Hazard ratio	Logarithm	Marshall et al. (2009)
Explained variance R^2	Fisher z on root	Harel (2009)
Survival probabilities	Complementary log-log	Marshall et al. (2009)
Survival distribution	Logarithm	Marshall et al. (2009)

parameter estimates $\hat{Q}$ are normally distributed around the population value Q with a variance of U . Many types of estimates are approximately normally distributed, e.g., means, standard deviations, regression coefficients, proportions and linear predictors. Rubin's pooling rules can be applied directly to such quantities (Schafer, 1997; Marshall et al., 2009).

5.2.2 Scalar inference of non-normal quantities

How should we combine quantities with non-normal distributions: correlation coefficients, odds ratios, relative risks, hazard ratios, measures of explained variance and so on? The quality of the pooled estimate and the confidence intervals can be improved when pooling is done in a scale for which the distribution is close to normal. Thus, transformation toward normality and back-transformation into the original scale improves statistical inference.

As an example, consider transforming a correlation coefficient ρ_ℓ for $\ell = 1, \dots, m$ toward normality using the Fisher z transformation

$$z_\ell = \frac{1}{2} \ln \frac{1 + \rho_\ell}{1 - \rho_\ell} \quad (5.1)$$

For large samples, the distribution of z_ℓ is normal with variance $\sigma^2 = 1/(n-3)$. It is straightforward to calculate the pooled correlation $\bar{z}$ and its variance by Rubin's rules. The result can be back-transformed by the inverse Fisher transformation

$$\bar{\rho} = \frac{e^{2\bar{z}} - 1}{e^{2\bar{z}} + 1} \quad (5.2)$$

The confidence interval of $\bar{\rho}$ is calculated in the z -scale as usual, and then back-transformed by Equation 5.2.

Table 5.2 suggests transformations toward approximate normality for various types of statistics. There are quantities for which the distribution is complex or unknown. Examples include the Cramér C statistic (Brand, 1999) and the discrimination index (Marshall et al., 2009). Ideally, the entire sampling distribution should be pooled in such cases, but the corresponding pooling methods have yet to be developed. The current advice is to search for ad hoc

transformations to make the sampling distribution close to normality, and then apply Rubin's rules.

5.3 Multi-parameter inference

There are many situations where we need to test whether a set of parameters is significantly different from zero. For example, if a categorical variable enters the analysis through a set of dummy variables, all parameters related to this set should be tested simultaneously. More generally, ANOVA type of designs can be formulated and tested as a multi-parameter inference regression problem. Van Ginkel and Kroonenberg (2014) provide many practical examples with missing data.

Schafer (1997) distinguished three types of statistics in multi-parameter tests: D_1 (multivariate Wald test), D_2 (combining test statistics) and D_3 (likelihood ratio test). The next sections outline the idea of each approach, and demonstrate how these tests can be performed as part of the repeated data analyses.

5.3.1 D_1 Multivariate Wald test

The multivariate Wald test is an extension of the procedure for scalar quantities as described in Section 2.4.2. The procedure tests whether $Q = Q_0$, where Q_0 is a k -vector of values under the null hypothesis (typically all zeros). The multivariate Wald test requires an estimate of the variance-covariance matrix U of $\bar{Q}$. We could use T from Equation 2.20, but this estimate can be unreliable. The problem is that for small m , the estimate of the between-imputation variance B is unstable, and if $m \leq k$, it is not even full rank. Thus T can be unreliable if B is a substantial component of T .

Li et al. (1991b) proposed an estimate of T in which B and $\bar{U}$ are assumed to be proportional to each other. A more stable estimate of the total variance is then $\tilde{T} = (1 + r_1)\bar{U}$, where $r_1 = \bar{r}$ (from Equation 2.29) is the average fraction of missing information.

Under the assumption that $(Q_0 - \bar{Q})$ is sufficiently close to a normal distribution, the D_1 -statistic

$$D_1 = (\bar{Q} - Q_0)' \tilde{T}^{-1} (\bar{Q} - Q_0) / k \quad (5.3)$$

follows an F -distribution F_{k, ν_1} with k and ν_1 degrees of freedom, where

$$\nu_1 = \begin{cases} 4 + (t - 4)[1 + (1 - 2t^{-1})r_1^{-1}]^2 & \text{if } t = k(m - 1) > 4 \\ t(1 + k^{-1})(1 + r_1^{-1})^2 / 2 & \text{otherwise} \end{cases} \quad (5.4)$$

The p -value for D_1 is

$$P_1 = \Pr[F_{k, \nu_1} > D_1] \quad (5.5)$$

The assumption that the fraction of missing information is the same across all variables and statistics is unlikely to hold in practice. However, Li et al. (1991b) provide encouraging simulation results for situations where this assumption is violated. Except for some extreme cases, the level of the procedure was close to the nominal level, while the loss of power from such violations was modest.

The work of Li et al. (1991b) is based on large samples. Reiter (2007) developed a small-sample version for the degrees of freedom using ideas similar to Barnard and Rubin (1999). Reiter's ν_f spans several lines of text, and is not given here. A simulation study conducted by Reiter showed marked improvements over the earlier formulation, especially in smaller samples. Simulation work by Grund et al. (2016b) and Liu and Enders (2017) confirmed that for small samples (say $n < 50$) ν_f is more conservative than ν_1 , and produced type I errors rates closer to their nominal value. Raghunathan (2015) recently provided an elegant alternative based on Equation 2.32 with ν_{obs} substituted as $\nu_{\text{obs}} = (\nu_{\text{com}} + 1)\nu_{\text{com}}/(\nu_{\text{com}} + 3)(1 + \bar{r})$. It is not yet known how this correction compares to ν_1 and ν_f .

The `mice` package implements the multivariate Wald test as the `D1()` function. Let us impute the `nhanes2` data, and fit the linear regression of `chl` on `age` and `bmi`.

```
library(mice)
imp <- mice(nhanes2, m = 10, print = FALSE, seed = 71242)
m2 <- with(imp, lm(chl ~ age + bmi))
pool(m2)
```

Class: mipo	m = 10							
	estimate	ubar	b	t	dfcom	df	riv	
(Intercept)	-2.98	2896.71	1266.44	4289.79	21	11.28	0.481	
age40-59	45.73	296.37	152.46	464.07	21	10.43	0.566	
age60-99	65.62	342.71	229.04	594.66	21	9.08	0.735	
bmi	6.40	3.26	1.41	4.81	21	11.32	0.477	
	lambda	fmi						
(Intercept)	0.325	0.419						
age40-59	0.361	0.456						
age60-99	0.424	0.519						
bmi	0.323	0.418						

We want to simplify the model by testing for `age`. Since `age` is a categorical variable with three categories, removing it involves deleting two columns at the same time, hence the univariate Wald test does not apply. The solution is to fit the model without `age`, and run the multivariate Wald statistic to test whether the model estimates are different.

```

m1 <- with(imp, lm(chl ~ bmi))
summary(D1(m2, m1))

Models:
  model      formula
    1 chl ~ age + bmi
    2      chl ~ bmi

Comparisons:
  test statistic df1  df2 df.com p.value  riv
1  ~~ 2         4.23   2 14.4    21   0.036 0.63

Number of imputations: 10    Method D1

```

Since the Wald test is significant, removing **age** from the model reduces its predictive power.

5.3.2 D_2 Combining test statistics♠

The multivariate Wald test may become cumbersome when k is large, or when many tests are to be done. Some analytic models may not produce an estimate of Q , or of its variance-covariance matrix. For example, nonparametric tests like the sign test or Wilcoxon-Mann-Whitney produce a p -value, and no estimate of Q . In cases like these, we can still calculate a combined significance test using the m test statistics (e.g., p -values, Z -values, χ^2 -values, t -values) as input.

Rubin (1987a, p. 87) and Li et al. (1991a) describe a procedure for pooling the values of the test statistics. Suppose that d_ℓ is the test statistic obtained from the analysis of the ℓ^{th} imputed dataset Y_ℓ , $\ell = 1, \dots, m$. Let $\bar{d} = m^{-1} \sum_\ell d_\ell$ be the average test statistic. The statistic for the combined test is

$$D_2 = \frac{\bar{d}k^{-1} - (m+1)(m-1)^{-1}r_2}{1 + r_2} \quad (5.6)$$

where the relative increase of the variance is calculated as

$$r_2 = \left(1 + \frac{1}{m}\right) \frac{1}{m-1} \sum_{\ell=1}^m \left(\sqrt{d_\ell} - \sqrt{\bar{d}}\right)^2 \quad (5.7)$$

with $\sqrt{\bar{d}} = m^{-1} \sum_\ell \sqrt{d_\ell}$, so that r_2 equals the sample variance of $\sqrt{d_1}, \sqrt{d_2}, \dots, \sqrt{d_m}$ multiplied by $(1 + 1/m)$. The p -value for testing the null hypothesis is

$$P_2 = \Pr[F_{k, \nu_2} > D_2] \quad (5.8)$$

where

$$\nu_2 = k^{-3/m} (m-1)(1 + r_2^{-1})^2 \quad (5.9)$$

The procedure assumes that the test statistic is approximately normally distributed. This is clearly not the case for p -values, which follow a uniform distribution under the null. One may transform the p -values to approximate normality, combine and back-transform afterwards. Based on this idea, Licht (2010, pp. 40–43) proposed a method for obtaining significance levels from repeated p -values similar to Equation 5.6 with custom r_2 and ν_2 .

In context of significance testing for logistic regression, Eekhout et al. (2017) suggest taking the median of the m p -values as the combined p -value, an exceedingly simple method. It nevertheless appears to outperform more sophisticated techniques if the variable to be tested is categorical with more than two categories. It would be useful to explore whether this *median P rule* has wider validity.

Let us continue with the previous example. Suppose that our software cannot export the variance-covariance matrix in each repeated analysis, but it does provide a table with the Wald statistics for testing **age**. The `D2` function calculates the D_2 -statistic and its degrees of freedom as

```
D2(m2, m1)
```

	test	statistic	df1	df2	df.com	p.value	riv
1	~~ 2	2.26	2	17.6	NA	0.134	1.81

In contrast to the previous analysis, observe that the D_2 -statistic is not significant at an α -level of 0.05. The reason is that the D_2 test is less informed by the data, and hence less powerful than the D_1 test.

5.3.3 D_3 Likelihood ratio test

The likelihood ratio test (Meng and Rubin, 1992) is designed to handle situations where one cannot obtain the covariance matrices of the complete-data estimates. This could be the case if the dimensionality of Q is high, which can occur with partially classified contingency tables. For large n the procedure is equivalent to the method of Section 5.3.1. The likelihood ratio test is the preferred method for testing random effects (Singer and Willett, 2003), and connects to global model fit statistics in structural equation models (Enders and Mansolf, 2018).

Let the vector Q contain the parameters of interest. We wish to test the hypothesis $Q = Q_0$ for some given Q_0 . The usual scenario is that we compare two models, one where Q can vary freely and one more restrictive model that constrains $Q = Q_0$.

The procedure for calculating the likelihood ratio test is as follows. First, estimate $\bar{Q}$ (for the full model) and $\bar{Q}_0$ (for the restricted model) from the m datasets by Rubin's rules. Calculate the value of the log-likelihood functions $l(\hat{Q}_\ell)$ (for the full model) and $l(\hat{Q}_{0,\ell})$ (for the restricted model), and determine

the average of the likelihood ratio tests across the m datasets, i.e.,

$$\hat{d} = m^{-1} \sum_{\ell} -2(l(\hat{Q}_{0,\ell}) - l(\hat{Q}_{\ell})) \quad (5.10)$$

Then re-estimate the full and restricted models, with their model parameters fixed to $\bar{Q}$ and $\bar{Q}_0$, respectively, and average the corresponding likelihood ratio tests as

$$\bar{d} = m^{-1} \sum_{\ell} -2(l(\bar{Q}_{0,\ell}) - l(\bar{Q}_{\ell})) \quad (5.11)$$

The test statistic proposed by Meng and Rubin (1992) is

$$D_3 = \frac{\bar{d}}{k(1 + r_3)} \quad (5.12)$$

where

$$r_3 = \frac{m+1}{k(m-1)}(\hat{d} - \bar{d}) \quad (5.13)$$

estimates the average relative increase in variance due to nonresponse. The quantity r_3 is asymptotically equivalent to $\bar{r}$ from Equation 2.29. The p -value for D_3 is equal to

$$P_3 = \Pr[F_{k,\nu_3} > D_3] \quad (5.14)$$

where $\nu_3 = \nu_1$, or equal Reiter's correction for small samples.

The likelihood ratio test does not require normality. For complete data, the likelihood ratio test is invariant to scale changes, which is the reason that many prefer the likelihood ratio scale over the Wald test. However, Schafer (1997, p. 118) observed that the invariance property is lost in multiple imputation because the averaging operations in Equations 5.10 and 5.11 may yield somewhat different results under nonlinear transformations of $l(Q)$. He advised that the best results will be obtained if the distribution of Q is approximately normal. One may transform the parameters to achieve normality, provided that appropriate care is taken to infer that the result is still within the allowable parameter space.

Liu and Enders (2017) found in their simulations that D_3 can become negative, a nonsensical value, in some scenarios. They suggest that a value of $r_3 > 10$ or a 1000% increase in sampling variance due to missing data may act as warning signals for this anomaly.

Routine use of the likelihood ratio statistic has long been hampered by difficulties in calculating the likelihood ratio tests for the models with fixed parameters $\bar{Q}$ and $\bar{Q}_0$. With the advent of the **broom** package (Robinson, 2017), the calculations have become feasible for a wide class of models. The `D3()` function in **mice** can be used to calculate the likelihood ratio test. We apply it to the data from previous examples by

D3(m2, m1)

	test	statistic	df1	df2	df.com	p.value	riv
1	~~	2	9.57	2	18	Inf	0.00147 766

The D_3 -statistic strongly indicates that **age** is a significant predictor. Note however the extremely large value for r_3 (column **riv**), so the result must be taken with a grain of salt. The likely cause for this anomaly could well be the lack of a small-sample correction for this test.

5.3.4 D_1 , D_2 or D_3 ?

If the estimates are approximately normal and if the software can produce the required variance-covariance matrices, we recommend using D_1 with an adjustment for small samples if $n < 100$. D_1 is a direct extension of Rubin's rules to multi-parameter problems, theoretically convincing, mature and widely applied. D_1 is insensitive to the assumption of equal fractions of missing information, is well calibrated, works well with small m (unless the fractions of information are large and variable) and suffers only modest loss of power. The relevant literature (Rubin, 1987a; Li et al., 1991b; Reiter, 2007; Grund et al., 2016b; Liu and Enders, 2017) is quite consistent.

If only the test statistics are available for pooling, then the D_2 -statistic is a good option, provided that the number of imputations $m > 20$. The test is easy to calculate and applies to different test statistics. For $m < 20$, the power may be low. D_2 tends to become optimistic for high fractions of missing information (> 0.3), and this effect unfortunately increases with sample size (Grund et al., 2016b). Thus, careless application of D_2 to large datasets with many missing values may yield high rates of false positives.

The likelihood ratio statistic D_3 is theoretically sound. Calculation of D_3 requires refitting the repeated analysis models with the estimates constrained to their pooled values. This was once an issue, but probably less so in the future. D_3 is asymptotically equivalent to D_1 , and may be preferred for theoretical reasons: it does not require normality in the complete-data model, it is often more powerful and it may be more stable than if k is large (as $\bar{U}$ need not be inverted). Grund et al. (2016b), Liu and Enders (2017) and Eekhout et al. (2017) found that D_3 produces Type 1 error rates that were comparable to D_1 . D_3 tends to be somewhat conservative in smaller samples, especially with high fractions of missing information and with high k . Also, D_3 has lower statistical power in some of the extreme scenarios. For small samples, D_1 has a slight edge over D_3 , so given the current available evidence D_1 is the better option for $n < 200$. In larger samples ($n \geq 200$) D_1 and D_3 appear equally good, so the choice between them is mostly a matter of convenience.

5.4 Stepwise model selection

The standard multiple imputation scheme consists of three phases:

1. Imputation of the missing data m times;
2. Analysis of the m imputed datasets;
3. Pooling of the parameters across m analyses.

This scheme is difficult to apply if stepwise model selection is part of the statistical analysis in phase 2. Application of stepwise variable selection methods may result in sets of variables that differ across the m datasets. It is not obvious how phase 3 should be done.

5.4.1 Variable selection techniques

Brand (1999, chap. 7) was the first to recognize and treat the variable selection problem. He proposed a solution in two steps. The first step involves performing stepwise model selection separately on each imputed dataset, followed by the construction of a new supermodel that contains all variables that were present in at least half of the initial models. The idea is that this criterion excludes variables that were selected accidentally. Moreover, it is a rough correction for multiple testing. Second, a special procedure for backward elimination is applied to all variables present in the supermodel. Each variable is removed in turn, and the pooled likelihood ratio p -value (Equation 5.14) is calculated. If the largest p -value is larger than 0.05, the corresponding variable is removed, and the procedure is repeated on the smaller model. The procedure stops if all $p \leq 0.05$. The procedure was found to be a considerable improvement over complete-case analysis.

Yang et al. (2005) proposed variable selection techniques using Bayesian model averaging. The authors studied two methods. The first method, called “impute then select,” applies Bayesian variable selection methods on the imputed data. The second method, called “simultaneously impute and select” combines selection and missing data imputation into one Gibbs sampler. Though the latter slightly outperformed the first method, the first method is more broadly applicable. Application of the second method seems to require equivalent imputation and analysis models, thus defeating one of the main advantages of multiple imputation.

Wood et al. (2008) and Vergouwe et al. (2010) studied several scenarios for variable selection. We distinguish three general approaches:

1. *Majority*. A method that selects variables in the final that appear in at least half of the models.

2. *Stack*. Stack the imputed datasets into a single dataset, assign a fixed weight to each record and apply the usual variable selection methods.
3. *Wald*. Stepwise model selection is based on the Wald statistic calculated from the multiply imputed data.

The majority method is identical to step 1 of Brand (1999), whereas the Wald test method is similar to Brand's step 2, with the likelihood ratio test replaced by the Wald test. The Wald test method is recommended since it is a well-established approach that follows Rubin's rules, whereas the majority and stack methods fail to take into account the uncertainty caused by the missing data. Indeed, Wood et al. (2008) found that the Wald test method is the only procedure that preserved the type I error.

Zhao and Long (2017) review recent work on variable selection on imputed data. These authors favor approaches based on the least absolute shrinkage and selection operator (LASSO) (Tibshirani, 1996). The MI-LASSO method by Chen and Wang (2013) tests the coefficients across all the stacked datasets, thus ensuring model consistency across different imputations. Marino et al. (2017) proposed an extension to select covariates in multilevel models.

In practice, it may be useful to combine methods. The Wald test method is computationally intensive, but is now easily available in `mice` as the `D1()` function. A strong point of the majority method is that it gives insight into the variability between the imputed datasets. An advantage of the stack method is that only one dataset needs to be analyzed. The discussion of Wood et al. (2008) contains additional simulations of a two-step method, in which a preselection made by the majority and stack methods is followed by the Wald test. This yielded a faster method with better theoretical properties. In practice, a judicious combination of approaches might turn out best.

5.4.2 Computation

The following steps illustrate the main steps involved by implementing a simple majority method to select variables in `mice`.

```
data <- boys[boys$age >= 8, -4]
imp <- mice(data, seed = 28382, m = 10, print = FALSE)
scope <- list(upper = ~ age + hgt + wgt + hc + gen + phb + reg,
             lower = ~1)
expr <- expression(f1 <- lm(tv ~ 1),
                  f2 <- step(f1, scope = scope))
fit <- with(imp, expr)
```

This code imputes the `boys` data $m = 10$ times, fits a stepwise linear model to predict `tv` (testicular volume) separately to each of the imputed dataset. The following code blocks counts how many times each variable was selected.


```
formulas <- lapply(fit$analyses, formula)
terms <- lapply(formulas, terms)
votes <- unlist(lapply(terms, labels))
table(votes)
```

```
votes
age gen  hc hgt phb reg wgt
10   9   1  6   9  10   1
```

The `lapply()` function is used three times. The first statement extracts the model formulas fitted to the m imputed datasets. The second `lapply()` call decomposes the model formulas into pieces, and the third call extracts the names of the variables included in all m models. The `table()` function counts the number of times that each variable in the 10 replications. Variables `age`, `gen` and `reg` are always included, whereas `hc` was selected in only one of the models. Since `hgt` appears in more than 50% of the models, we can use the Wald test to determine whether it should be in the final model.

```
fit.without <- with(imp, lm(tv ~ age + gen + reg + phb))
fit.with <- with(imp, lm(tv ~ age + gen + reg + phb + hgt))
D1(fit.with, fit.without)
```

```
      test statistic df1  df2 df.com p.value   riv
1  ~~ 2          2.15   1 19.3   409  0.159 0.978
```

The p -value is equal to 0.173, so `hgt` is not needed in the model. If we go one step further, and remove `phb`, we obtain

```
fit.without <- with(imp, lm(tv ~ age + gen + reg))
fit.with <- with(imp, lm(tv ~ age + gen + reg + phb))
D1(fit.with, fit.without)
```

```
      test statistic df1  df2 df.com p.value   riv
1  ~~ 2          2.49   5 97.9   410  0.0362 1.29
```

The significant difference ($p = 0.029$) between the models implies that `phb` should be retained. We obtain similar results for the other three variables, so the final model contains `age`, `gen`, `reg` and `phb`.

5.4.3 Model optimism

The main danger of data-driven model building strategies is that the model found may depend highly on the sample at hand. For example, Viallefont et al. (2001) showed that of the variables declared to be “significant” with p -values between 0.01 and 0.05 by stepwise variable selection, only 49% actually were true risk factors. Various solutions have been proposed to counter

such *model optimism*. A popular procedure is bootstrapping the model as developed in Sauerbrei and Schumacher (1992) and Harrell (2001). Although Austin (2008) found it ineffective to identify true predictors, this method has often been found to work well for developing predictive models. The method randomly draws multiple samples with replacement from the observed sample, thus mimicking the sampling variation in the population from which the sample was drawn. Stepwise regression analyses are replayed in each bootstrap sample. The proportion of times that each prognostic variable is retained in the stepwise regression model is known as the *inclusion frequency* (Sauerbrei and Schumacher, 1992). This proportion provides information about the strength of the evidence that an indicator is an independent predictor. In addition, each bootstrap model can be fitted to the original sample. The difference between the apparent performance and the bootstrap performance provides the basis for performance measures that correct for model optimism. Steyerberg (2009, p. 95) provides an easy-to-follow procedure to calculate such *optimism-corrected performance* measures.

Clearly, the presence of missing data adds uncertainty to the model building process, so optimism can be expected to be more severe with missing data. It is not yet clear what the best way is to estimate optimism from incomplete data. Heymans et al. (2007) explored the combination of multiple imputation and the bootstrap. There appear to be at least four general procedures:

1. *Imputation*. Multiple imputation generates 100 imputed datasets. Automatic backward selection is applied to each dataset. Any differences found between the 100 fitted models are due to the missing data.
2. *Bootstrap*. 200 bootstrap samples are drawn from one singly imputed completed data. Automatic backward selection is applied to each dataset. Any differences found between the 200 fitted models are due to sampling variation.
3. *Nested bootstrap*. The bootstrap method is applied on each of the multiply imputed datasets. Automatic backward selection is applied to each of the 100×200 datasets. Differences between the fitted model portray both sampling and missing data uncertainty.
4. *Nested imputation*. The imputation method is applied on each of the bootstrapped datasets.

Heymans et al. (2007) observed that the imputation method produced a wider range of inclusion frequencies than the bootstrap method. This is attractive since a better separation of strong and weak predictors may ease model building. The area under the curve is an overall index of predictive strength. Though the type of method had a substantial effect on the apparent *c*-index estimate, the optimism-corrected *c*-index estimate was quite similar. The optimism-corrected calibration slope estimates tended to be lower in the methods involving imputation, thus necessitating more shrinkage.

A drawback of the method is the use of classic stepwise variable selection techniques, which do not generalize well to high-dimensional data. Musoro et al. (2014) improved the methods of Heymans et al. (2007) through their use of the LASSO.

Long and Johnson (2015) developed a procedure, called bootstrap imputation and stability selection (BI-SS), that generates bootstrap samples from the original data, imputes each bootstrap sample by single imputation, obtains the randomized LASSO estimate from each sample, and then selects the active set according to majority. The multiple imputation random LASSO (MIRL) method by Liu et al. (2016) first performs multiple imputation, obtains bootstrap samples from each imputed dataset, estimates regression weights under LASSO, and then selects the active set by majority. It is not yet known how BS-SS and MIRL compare to each other.

5.5 Parallel computation

Multiple imputation is a parallel technique. If there are m processors available, it is possible to generate the m imputed datasets, estimate the m complete-data statistics and store the m results by m independent parallel streams. The overhead needed is minimal since each stream requires the same amount of processor time. If more than m processors are available, a better alternative is to subdivide each stream into several substreams. Huge savings in execution time can be obtained in this way (Beddo, 2002).

Unfortunately, R is single-threaded, so the exploitation of the parallel nature of multiple imputation is not automatic, and requires some additional work. There are currently three alternatives to perform the calculation of `mice` in a parallel fashion.

1. Gordon (2014) presents a fully worked out example code that builds upon the `doParallel` library, and that combines `complete()` and `ibind()`. With some programming this example can be adapted to other datasets.
2. The `parLMICE()` function is a wrapper around `mice()` that can divide the imputations over multiple cores or CPUs. Schouten and Vink (2017) show that substantial gains are already possible with three free cores, especially for a combination of a large number of imputations m and a large sample size n .
3. The `par.mice()` function in the `micemd` package (Audigier and Resche-Rigon, 2018) takes the same arguments as the `mice()` function, plus two extra arguments related to the parallel calculations. It also builds on the `parallel` package.

The last two options are quite similar. Application of these methods is especially beneficial for simulation studies, where the same model needs to be replicated a large number of times. Support for multi-core processing is likely to grow, so keep an eye on the Internet.

5.6 Conclusion

The statistical analysis of the multiply imputed data involved repeated analysis followed by parameter pooling. Rubin's rules apply to a wide variety of quantities, especially if these quantities are transformed toward normality. Dedicated statistical tests and model selection technique are now available. Although many techniques for complete data now have their analogues for incomplete data, the present state-of-the-art does not cover all. As multiple imputation becomes more familiar and more routine, we will see new post-imputation methodology that will be progressively more refined.

5.7 Exercises

Allison and Cicchetti (1976) investigated the interrelationship between sleep, ecological and constitutional variables. They assessed these variables for 39 mammalian species. The authors concluded that slow-wave sleep is negatively associated with a factor related to body size. This suggests that large amounts of this sleep phase are disadvantageous in large species. Also, paradoxical sleep was associated with a factor related to predatory danger, suggesting that large amounts of this sleep phase are disadvantageous in prey species.

Allison and Cicchetti (1976) performed their analyses under complete-case analysis. In this exercise we will recompute the regression equations for slow wave ("nondreaming") sleep (hrs/day) and paradoxical ("dreaming") sleep (hrs/day), as reported by the authors. Furthermore, we will evaluate the imputations.

1. *Complete-case analysis.* Compute the regression equations (1) and (2) from the paper of Allison and Cicchetti (1976) under complete-case analysis.
2. *Imputation.* The `mammalsleep` data are part of the `mice` package. Impute the data with `mice()` under all the default settings. Recalculate the regression equations (1) and (2) on the multiply imputed data.

3. *Traces.* Inspect the trace plot of the MICE algorithm. Does the algorithm appear to converge?
4. *More iterations.* Extend the analysis with 20 extra iterations using `mice.mids()`. Does this affect your conclusion about convergence?
5. *Distributions.* Inspect the data with diagnostic plots for univariate data. Are the univariate distributions of the observed and imputed data similar? Can you explain why they do (or do not) differ?
6. *Relations.* Inspect the data with diagnostic plots for the most interesting bivariate relations. Are the relations similar in the observed and imputed data? Can you explain why they do (or do not) differ?
7. *Defaults.* Consider each of the seven default choices from [Section 6.1](#) in turn. Do you think the default is appropriate for your data? Explain why.
8. *Improvement.* Do you have particular suggestions for improvement? Which? Implement one (or more) of your suggestions. Do the results now look more plausible or realistic? Explain. What happened to the regression equations?
9. *Multivariate analyses.* Repeat the factor analysis and the stepwise regression. Beware: There might be pooling problems.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>